
Assignment: Aviation Ticket Pricing Engine

Course Module: Advanced C# Concepts – Delegates & Functional Paradigms

Objective

Apply C# delegates to build a flexible, decoupled pricing engine. You will trace the historical evolution of C# by implementing dynamic business logic using Named Methods, Anonymous Methods, and Lambda Expressions, strictly adhering to the Open-Closed Principle (OCP).

Part 1: Environment Setup

Design the core domain model for the aviation system.

1. Create a class named `FlightTicket` with the following properties:
 - `TicketId` (integer)
 - `PassengerName` (string)
 - `BasePrice` (decimal)
 - `FlightTime` (DateTime)
 - `IsDelayed` (boolean)
2. In your main execution entry point, initialize a `List<FlightTicket>` containing at least 5 tickets with varying data.

Part 2: The Named Methods

Design a pricing engine that applies different discount rules without modifying the core processing logic.

1. Define a delegate named `DiscountCalculator` that takes a `FlightTicket` as a parameter and returns a `decimal` (representing the final price).
2. Create a method named `ProcessTickets` that accepts a `List<FlightTicket>` and a `DiscountCalculator` delegate. This method should iterate through the tickets and print the passenger's name, original price, and the final discounted price.
3. Create a separate class named `DiscountRules` to house your business logic. Add the following methods:
 - A static method `DelayedFlightDiscount`: Applies a 20% discount if the flight is delayed.
 - An instance method `VipPassengerDiscount`: Applies a flat 15% discount unconditionally.
4. Pass both methods to the `ProcessTickets` engine to verify that the delegate accepts both static and instance methods as long as the signature matches.

Part 3: Transition to Anonymous Methods

The marketing team requested a one-time "Holiday Special" discount of 10% for all tickets.

Instead of cluttering the `DiscountRules` class with a method that will only be used once, invoke `ProcessTickets` using an Anonymous Method `delegate(FlightTicket t) { ... }` directly inline.

Part 4: The Lambda Expressions

The business requires two new dynamic discounts. Implement them using Lambda Expressions to write concise, modern C# code:

- 1 **Morning Flight Discount:** A 5% discount for flights scheduled before 12:00 PM (Noon).
- 2 **Premium Ticket Discount:** A flat \$50 deduction for any ticket where the `BasePrice` exceeds \$1,000.

Pass these logic rules directly into `ProcessTickets` using the `t => ...` syntax.

Part 5: Bonus Challenge (Multicast Delegates)

Optional — for advanced exploration.

Delegates can point to multiple methods simultaneously, forming the foundation of event-driven programming.

1. *Define a delegate `NotificationSender` that takes a `FlightTicket` and returns `void`.*
2. *Create two methods: `SendEmail` and `SendSMS`, which print out corresponding notification messages for the passenger.*
3. *Instantiate the delegate and use the `+=` operator to attach both methods to it.*
4. *Invoke the delegate once and verify that both the Email and SMS functions are executed consecutively.*

End of Assignment — Please submit your completed solution as a single .cs file or a zipped Visual Studio project.